



Prueba Técnica – Backend Developer Python

Contexto

En la empresa recibimos archivos Excel en formato .xlsx con información que debe ser cargada, validada y consultada desde una API.

En el siguiente enlace encontrara un archivo Excel que contiene:

- Una primera hoja con el template de datos que el usuario debe diligenciar.
- Una segunda hoja con las restricciones o reglas de validación de cada campo.

[Template](#)

El objetivo es construir una solución backend que permita subir el archivo, validar la información y consultar los datos cargados.

Objetivo

Desarrollar una API backend en Python que permita:

1. Cargar un archivo .x/sx.
2. Leer los registros del archivo (para eso usar el template).
3. Validar cada registro contra las reglas definidas.
4. Guardar los registros válidos.
5. Reportar claramente los errores encontrados.
6. Consultar los datos cargados mediante un endpoint paginado.



Requisitos funcionales

La API debe incluir como mínimo:

Endpoint para cargar archivo

Debe permitir subir un archivo .x/sx.

El sistema debe validar:

- Que el archivo sea formato .x/sx
- Que los encabezados del template sean correctos.
- Que cada fila cumpla las restricciones definidas.

Por cada fila inválida, se debe reportar:

- Número de fila.
- Campo con error.
- Valor recibido
- Mensaje del error.

Ejemplo de respuesta:

```
{ "total_rows": 50, "valid_rows": 45, "invalid_rows": 5, "errors": [ { "row": 7, "field": "email", "value": "correo-invalido", "message": "Invalid email format" } ] }
```

Validación de registros

La solución debe soportar validaciones como:

- Campo requerido.
- Tipo de dato esperado.



- Longitud mínima o máxima.
- Valores permitidos.
- Formato de email.
- Valores numéricos positivos.
- Fechas válidas.

No es obligatorio soportar todas las validaciones posibles, pero sí se debe implementar una estructura que permita extenderlas fácilmente.

Persistencia

Los registros válidos deben guardarse en una base de datos.

Puedes usar PostgreSQL, SQLite u otra base de datos relacional.

Endpoint paginado de consulta

Debe existir un endpoint para consultar los registros cargados.

Debe soportar paginación, por ejemplo: GET /records?page=1&page_size=20

Respuesta esperada:

```
{ "page": 1, "page_size": 20, "total": 100, "items": [] }
```



Requisitos técnicos

Se espera el uso de:

- Python.
- FastAPI, Flask o Django REST Framework.
- Pydantic o validaciones equivalentes.
- SQLAlchemy, SQLModel, Django ORM o equivalente.
- Pruebas automatizadas.
- README con instrucciones claras.

Se valorará positivamente:

- Uso de Docker.
- Migraciones con Alembic.
- Separación clara entre API, lógica de negocio y persistencia.
- Buen manejo de errores.
- Código limpio y mantenible.
- Tests unitarios y/o de integración.

Entrega

La entrega debe incluir:

- Código fuente completo en github o en un control de versiones en la nube similar.



- README con instrucciones para ejecutar el proyecto
- Archivo de ejemplo .xlsx usado para pruebas.
- Pruebas automatizadas.
- Archivo de dependencias (requirements, pyproject.toml o el recomendado)
- Instrucciones para correr la base de datos si aplica.

Criterios de evaluación

Se evaluará:

- Correcta lectura del archivo Excel.
- Validación clara de los registros.
- Reporte entendible de errores.
- Persistencia de registros válidos.
- Endpoint paginado funcional.
- Diseño de arquitectura.
- Calidad y legibilidad del código.
- Cobertura básica de pruebas.
- Capacidad para explicar y modificar la solución.

No buscamos una solución perfecta. Buscamos entender cómo analizas, diseñas, desarrollas, pruebas y explicas una solución backend realista.